Machine Learning Algorithms in Neural Networks

Linéa Lejerkrans and Clara Gomer

***Abstract*—This project explores the theoretical foundations and practical implementation of feedforward neural networks. A simple network was implemented and trained on the MNIST data set using stochastic gradient descent combined with backpropagation. Training performance was analyzed using two different cost functions, cross-entropy and quadratic, as well as L2 regularization and limited tuning of hyperparameters. As expected, carefully selecting the learning rate significantly improved convergence. Furthermore, L2 regularization proved to be an effective method for reducing overfitting and improving validation accuracy, leading to a final classification accuracy of 96.4% when using the cross-entropy cost function. The results of the implementations also highlight the fact that theoretical expectations might require adjustments in practice.**

***Sammanfattning*—Detta projekt utforskar de teoretiska grunderna och den praktiska implementeringen av framåtriktade neurala nätverk. Ett enkelt nätverk implementerades och tränades på MNIST-datasettet med hjälp av stokastisk gradientnedstigning kombinerat med bakåtpropagering. Träningsprestandan analyserades med två olika kostnadsfunktioner, cross-entropy och kvadratisk, samt med L2-regularisering och begränsad justering av hyperparametrar. Som förväntat förbättrades konvergensen avsevärt genom ett noggrant val av inlärningshastighet. L2-regularisering visade sig dessutom vara en effektiv metod för att minska överanpassning och förbättra valideringsnoggrannheten, vilket resulterade i en högsta klassificeringsnoggrannhet på 96.4%. Resultaten från implementeringarna påvisar också att teoretiska förväntningar ofta kräver justeringar i praktiken.**

***Index Terms*—neural networks, classification, gradient descent, backpropagation, regularization**

Supervisors: Seyedamirreza Kazemi and Carlo Fischione

TRITA number: TRITA-EECS-EX-2025:142

I. Introduction

Neural networks are computational systems inspired by the human brain, designed to recognize patterns, make predictions, and solve complex tasks without relying on explicitly programmed rules. Through a training process that adjusts the internal parameters of the network, the model learns to map input data to desired outputs. This ability to learn directly from data makes neural networks especially powerful for solving problems that are difficult to model explicitly. These networks have become the foundation of modern machine learning, with applications ranging from image and speech recognition to game play and autonomous systems.

A. Background

The development of neural networks has a history dating back to the early 1940s. Using electrical circuits, neurophysiologist Warren McCulloch and mathematician Walter Pitts modeled a simple neural network to better understand how neurons in the human brain work [1]. In 1949, Donald Hebb introduced the fundamental concept of the learning process, in which neural pathways are strengthened each time the network is used. As computers became more advanced, it was possible to simulate a hypothetical neural network. Models like ”ADALINE” and ”MADALINE”, developed by Bernard Widrow and Marcian Hoff, were applied to real-world problems, such as eliminating echoes on telephone lines [1]. However, due to theoretical limitations along with the rising popularity of von Neumann computing architecture-which is based on the concept of storing program instructions and data in the same memory and processing them sequentially [2] research on neural networks stagnated. In 1980, due to advances and research exploring hybrid networks that combined multiple strategies, interest was revived [1]. In 1986, a major breakthrough occurred when three independent research groups came up with a similar idea, now known as backpropagation. Although backpropagation networks are slow learners, they laid the foundation of deep learning.

B. Aim and Scope

The purpose of this project is to explore the theoretical principles of neural networks and try to validate them through simple implementations. A feedforward neural network was implemented and trained on the MNIST dataset using stochastic gradient descent. Several key aspects were investigated, including the effects of different cost functions (quadratic vs. cross-entropy), the tuning of the learning rate (step size), and the impact of L2 regularization to reduce overfitting. However, the implementation was intentionally kept simple to keep focus on the fundamental characteristics rather than creating a highly performing network. Not all hyperparameters, such as the number of neurons per layer, batch size, or number of hidden layers, were optimized. The tuning of hyperparameters was done manually and was limited in scope, without using systematic methods like grid search or random search. Subsequently, while the project highlights important behavior and trends in a neural network, it does not aim to achieve optimal results on the MNIST data set.

This report is structured as follows. Section 2 presents the theoretical background of neural networks and training methods. Section 3 describes the implementation process. Section 4 discusses the experimental results in relation to theoretical expectations. Section 5 concludes the report and highlights key takeaways, while Section 6 suggests possible directions for future work.

II. Theory

A neural network consists of layers of interconnected ”neurons”, which can be seen in Figure 1. The input layer is an

entry point for information and does not perform any computations. The hidden layers performs a series of mathematical operations and passes the output to the next layer. Finally, the output layer provides the results [3].

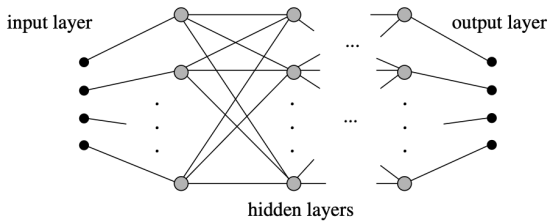


Fig. 1. Architecture of a multilayer perceptron (MLP) showing the input layer, one or more hidden layers and output layer. Each layer consists of interconnected nodes (neurons) that process and pass information forward through the network.

A. Artificial Neurons

As mentioned above, neural networks are designed to mimic the function of the human brain, particularly how neurons process information and make decisions. To achieve this, artificial neurons are used. One of the simplest types is called a perceptron. It takes multiple binary inputs and produces one binary output [4].

The output is determined using a weight w_j and a bias b. The weights are real numbers that represent the importance of each input in influencing the output. The bias controls how easily the perceptron produces an output of 1 [4].

$$\text{output} = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases} \tag{1}$$

The problem with perceptrons is that they can only produce 0 or 1, which means that a small change in w can have a major impact on the output [3]. Therefore, other activation functions, such as the sigmoid function (σ), are more commonly used in artificial neurons.

$$\sigma(z) = \frac{1}{1+e^{-z}}; \quad z = \sum_j w_j x_j + b \tag{2}$$

Unlike perceptrons, they can produce continuous values instead of just binary ones. The sigmoid function outputs values between 0 and 1, [3] [5].

These artificial neurons allows for a more controlled learning because a small change in weights results in only a small change in the network's output. This allows the network to be continuously trained by adjusting the weights and biases, gradually improving its results [5].

B. Cost Function

To evaluate how well the output of the network a compares to the right answer $y(x)$, a cost function is defined as $C(w, b)$. The following is the definition of the quadratic cost function. In later subsections, other more efficient cost functions will be used [4].

$$C(w,b) \equiv \frac{1}{2n} \sum_x \|y(x) - a\|^2 \tag{3}$$

The weight and bias should be adjusted so that the magnitude of the cost function decreases over time. It is difficult to directly maximize the number of correctly classified images because classification accuracy is a non-smooth function of the weights and biases. A small change in weight and bias will not necessarily increase the number of correctly classified images. This makes it difficult to determine how to adjust the weights and biases to improve performance. Instead, the focus lies on minimizing the cost function, which provides a smooth function of error [4].

C. Gradient Descent

The cost function can be minimized using an optimization algorithm known as gradient descent. By repeatedly computing the gradient of the cost function ∇C, and then moving in the opposite direction, the function can be minimized [5]. This approach is effective because the gradient indicates the direction of the steepest increase in C, allowing a systematic reduction in cost [6].

Definition of the gradient ∇, [3]:

$$\nabla C = \left(\frac{\partial C}{\partial x_1}, \frac{\partial C}{\partial x_2}, \ldots, \frac{\partial C}{\partial x_n} \right) \tag{4}$$

Description of how the cost function changes depending on the different parameters [4]:

$$\Delta C = \frac{\partial C}{\partial x_1} \Delta x_1 + \frac{\partial C}{\partial x_2} \Delta x_2 + \ldots, \frac{\partial C}{\partial x_n} \Delta x_n \tag{5}$$

Equations (4) and (5) give:

$$\Delta C = \nabla C \cdot \Delta x \tag{6}$$

Δx is chosen as $-\eta \nabla C$ [4]. Here η is a small positive parameter (known as the learning rate) that defines how big the steps in each iteration should be [5], [3]. Choosing Δx like this also ensures that ΔC is always negative: $||\nabla C|\,|^2 \geq 0$ [4].

Applied to a neural network, these equations show how the weight and bias should behave to decrease the cost function [4].

$$w_k' = w_k - \eta \frac{\partial C}{\partial w_k} \tag{7}$$

$$b_l' = b_l - \eta \frac{\partial C}{\partial b_l} \tag{8}$$

This process will be repeated until the global minimum is found, which means that the optimal parameters for the network are acquired [6].

However, there are some challenges with this method. One challenge is that if the learning rate η is too large, it might cause overshooting and the minimum will be missed. On the other hand, if the steps are too small, the learning efficiency will decrease [4].

Another problem arises when the training data is very large. In such cases, computing the gradient for each input and

averaging them can be very time-consuming. A solution to this is stochastic gradient descent (SGD) [4]. The idea is to only compute the gradient using a smaller, randomly chosen subset of inputs, called a mini-batch, rather than using the entire data set [6]. After processing one mini-batch, another is randomly selected, and the process is repeated until the entire dataset has been processed once. This complete pass through the data set is known as an epoch. These computations will not be perfect, but they will provide a good enough approximation of the gradient to help decrease the cost function, which is the goal with the approach.

An estimated gradient, computed by calculating gradients for a randomly chosen mini-batch, can look like this [4]:

$$\nabla C \approx \frac{1}{m} \sum_{j=1}^{m} \nabla C_{x_j} \tag{9}$$

where m is a randomly chosen number of training inputs.

D. Backpropagation

Backpropagation is a popular method for computing gradient descent. It provides a detailed understanding of how changes in weights and biases affect the overall behavior of a neural network [5].

The activation of the j-th neuron in the l-th layer, a_j^l, is related to the activations in the $(l-1)$-th layer by this equation:

$$a_j^l = \sigma\left(\sum_k w_{jk}^l a_k^{l-1} + b_j^l\right), \tag{10}$$

where $w_{ij}^{(l)}$ represents the weight that connects the neuron j in layer l to the neuron i in layer $l+1$, and $b_i^{(l)}$ is the bias [5]. When calculating $a_i^{(l+1)}$, an intermediate quantity $z_i^{(l+1)}$, known as the weighted input, is computed along with [4]:

$$z_i^{(l+1)} = \sum_j w_{ij}^{(l)} a_j^{(l)} + b_i^{(l)} \tag{11}$$

To apply backpropagation to the neural network, two conditions must be met. First, the cost function must be decomposable, which means that the gradient can be computed independently for each sample and then averaged over the mini-bach [3]:

$$C = \frac{1}{n} \sum C_i \tag{12}$$

The second condition requires that the cost function be differentiable, allowing the computation of partial derivatives with respect to the weights and biases [3]. This means that the cost function can be written as a function of the output:

$$C = C(a_L) \tag{13}$$

Backpropagation gives an insight to how changes in weights and biases affect the cost function, enabling the model to learn from errors. To compute partial derivatives, an intermediate quantity, δ_j^l, is introduced, representing the error in the j-th neuron of the l-th layer [5]. This measures how the cost function changes with a small change in the weighted input z^l.

A small change in z^l affects the activations of all subsequent layers. This small change propagates through the network, affecting the activation of all other layers and ultimately changing the final cost by the amount [4]:

$$\frac{\partial C}{\partial z_j^l} \Delta z_j^l \tag{14}$$

There are four fundamental backpropagation equations that describe how the error propagates through the network. The first equation defines the error in the output layer as [4]:

$$\delta_j^L = \frac{\partial C}{\partial a_j^L} \sigma'(z_j^L), \tag{15}$$

where $\sigma'(z_j^L)$ represents the derivative of the activation function.

The second equation represents the error in terms of the next layer [4]:

$$\delta^l = (w^{l+1})^T \delta^{l+1} \odot \sigma'(z^l), \tag{16}$$

allowing backpropagation of errors from one layer to the previous.

The third and fourth equations define the gradients with respect to the biases as:

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l, \tag{17}$$

and the weights as [4]:

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l \tag{18}$$

These equations apply to any activation function, not just sigmoid [5]. Choosing an activation function where σ' never approaches zero prevents the slowdown in learning that occurs when sigmoid neurons saturate.

The learning rate of a weight is influenced by two key factors; small activation and saturation. If the input neuron's activation a_k^{l-1} is small (low activation), the weight update Δw_{jk}^l will also be small. This means that the weight learns slowly because it does not receive a strong signal from the input neuron [4].

If the output neuron is in a saturated region (where $\sigma(z)$ is close to 0 or 1 for sigmoid activation functions), the derivative $\sigma'(z)$ becomes very small. This causes δ_j^l to shrink, leading to a small weight update Δw_{jk}^l. In other words, if the output neuron is saturated (with high or low activation), the gradient is small, and the weight learns slowly [4].

E. Cross-entropy Cost Function

Previously, the quadratic cost function was used, but it has a significant drawback; it learns very slowly when the output is highly incorrect. This is because the neuron updates the weights and biases at a rate determined by the partial derivatives of the cost function, $\frac{\partial C}{\partial w}$ and $\frac{\partial C}{\partial b}$. Slow learning

occurs when these derivatives are small, often due to the near-flat nature of the sigmoid function at extreme values [5]. To address this, the cross-entropy cost function is introduced [4]:

$$C = -\frac{1}{n}\sum_x [y \ln a + (1-y) \ln (1-a)] \tag{19}$$

This cost function has several key properties. Firstly, it is always non-negative. Additionally, when the output is close to the desired value, the contribution to the total cost is low, allowing faster learning. It also avoids the slowed down learning that occurs when the sigmoid function saturates. Instead, the rate at which the weight is updated is directly proportional to the error in the output, $\sigma(z) - y$, which is desirable. This property can be demonstrated by calculating the partial derivative of the cross-entropy function with respect to the weights [4]:

$$\begin{aligned}\frac{\partial C}{\partial w_j} &= -\frac{1}{n}\sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)}\right)\frac{\partial \sigma}{\partial w_j} \\ &= -\frac{1}{n}\sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)}\right)\sigma'(z)x_j\end{aligned} \tag{20}$$

By putting everything over a common denominator, the following expression is obtained:

$$\frac{\partial C}{w_j} = \frac{1}{n}\sum_x \frac{\sigma'(z)x_j}{\sigma(z)(1-\sigma(z))}(\sigma(z) - y) \tag{21}$$

Using the definition of the sigmoid function, (equation 2) along with some algebra, the derivative can be written in terms of itself as:

$$\sigma'(z) = \sigma(z)(1-\sigma(z)) \tag{22}$$

By inserting this in equation 21 it is now proven that the partial derivative of the cross-entropy function with respect to the weight no longer involves $\sigma'(z)$ [4]:

$$\frac{\partial C}{\partial w_j} = \frac{1}{n}\sum_x x_j(\sigma(z) - y) \tag{23}$$

Expression regarding the bias:

$$\frac{\partial C}{\partial b} = \frac{1}{n}\sum_x (\sigma(z) - y) \tag{24}$$

The cross-entropy cost function acts as a measure of 'surprise': low surprise occurs when the value matches expectations, and high surprise occurs when the output is unexpected. This ensures that the network learns more quickly in the early stages when the output is far from the target, and avoids the problem of slow learning caused by the limitations of the quadratic cost function [4].

F. Overfitting

Although the cross-entropy cost function improves learning efficiency, one of the biggest problems in training neural networks still remains unaddressed; overfitting. Overfitting occurs when a neural networks cost function on the training data continues to decrease, suggesting ongoing improvement, while the classification accuracy on the validation data stabilizes [6]. This indicates that the network is memorizing the training data rather than learning useful patterns that generalize to unseen data.

A common solution for overfitting is early stopping. This technique involves monitoring the model's performance on a validation dataset during training and halting the process once the validation accuracy ceases to improve [5] [6]. By doing so, the model is prevented from overfitting to the training data.

Implementing early stopping requires dividing the data into training, validation, and testing sets. The model is trained on the training set, while its performance is evaluated on the validation set [6]. Training continues until validation accuracy stabilizes or declines, indicating that further training may lead to overfitting. At this point, training is stopped, and the model parameters corresponding to the highest validation accuracy are retained [5]. This prevents the network from overfitting the hyperparameters to the validation data.

Determining hyperparameters using the validation set ensures that the test set remains a true measure of the model's ability to generalize to unseen data. This approach is known as the holdout method, as the validation data is 'held out' from the training process [6].

G. Regularization

Regularization refers to techniques used to improve generalization and prevent overfitting [6]. The best way to reduce overfitting is to increase the size of the training data, but this can be expensive, so there are other methods too.

1) L2: The regularization method L2 adds an extra term to the cost function, called the regularization term. The regularized cross-entropy cost function can be written as [4]:

$$C = -\frac{1}{n}\sum_x [y_j \ln a_j^L + (1-y_j) \ln (1-a_j^L)] + \frac{\lambda}{2n}\sum_w w^2 \tag{25}$$

or

$$C = C_0 + \frac{\lambda}{2n}\sum_w w^2 \tag{26}$$

where C_0 is the original, unregularized cost function. The idea is to make the network prefer small weights, where large weights will only be allowed if they considerably improve the first part of the cost function. This technique compromises between finding small weights and minimizing the original cost function [6]. The strength of regularization is controlled by the regularization parameter λ, which regulates the weight decay effect. A large λ prefers small weights, while a small λ prioritizes minimizing the cost function [4].

Because these regularization techniques keep the weights small, the behavior of the network remains stable even when

a few random inputs deviate from the overall pattern. This prevents individual data pieces from having too much influence on the output, encouraging the network to learn patterns that appear consistently throughout the training set, rather than noise [6]. In contrast, networks with large weights can be highly sensitive to small input changes, increasing their likelihood of overfitting to noise.

2) L1: The regularization method L1 adds a different term to the cost function compared to L2. The regularized cross-entropy function can be written as [4]:

$$C = C_0 + \frac{\lambda}{n} \sum_w |w| \tag{27}$$

Weight updates with the L1 technique:

$$w \to w' = w - \frac{\eta\lambda}{n} \text{sgn}(w) - \eta \frac{\partial C_0}{\partial w} \tag{28}$$

This technique is similar to regularization L2, but the way the weights decreases differs. L1 regularization tends to produce a sparse weight vector, with many components driven exactly to zero, while when using L2, the weights decrease proportionally to their magnitude [6]. This means that for large weights, L2 reduces weights more aggressively than L1, while for small weights, L1 decreases them more quickly compared to L2.

3) Dropout: Another way of regularizing the network is using the dropout method. Dropout involves randomly deactivating half of the neurons in the hidden layers during training. In each mini-batch, the weights and biases are updated as usual, and then new neurons are randomly selected for deactivation. This process is repeated throughout training, with a different set of neurons being dropped out each time [6]. When the full network is run, twice as many hidden neurons are active, so to compensate, the outgoing weights from the hidden neurons are scaled by half.

Essentially, dropout trains multiple different networks simultaneously and by combining their outputs, each of which overfits in different ways, it helps reduce overall overfitting [4].

4) Invariance: Reference [5] shows that another efficient way to improve generalization is invariance. Certain transformations of the input data should not affect the output prediction in recognition tasks. For example, in digit classification, a digit should be recognized regardless of position, size or orientation. However, neural networks do not possess this ability naturally. Although larger data sets might deal with invariance simply by encountering many examples of each transformation, the training data are often limited in practice. A commonly used approach is to expand the training data set artificially by applying transformations such as rotations, translations, and scaling (for example, for MNIST images) [6]. Adding noise or denoising before training is also an efficient way of reducing overfitting.

H. Weight Initialization

Proper initialization helps avoid problems such as neuron saturation and improves the efficiency of the network [3].

If weight and bias come from a normal (Gaussian) distribution, the sum z will also follow a normal distribution, with a mean of 0 and a standard deviation based on the total number of terms (for example, $n = 500 \to \sqrt{500} \approx 22.4$) [5]. This means that z has a broad spread, which can be seen in Figure 2 [4].

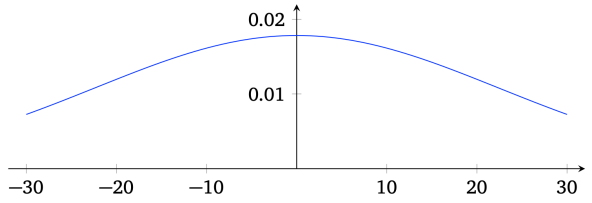


Fig. 2. A normal (Gaussian) distribution with mean 0 and a broad standard deviation, illustrating the spread of the weighted sum z when weights are initialized from the same distribution.

The graph shows that it is very likely that z is large, resulting in the output of the hidden neuron, $\sigma(z)$, being very close to either 1 or 0, meaning that the hidden neurons will have saturated [4].

An optimized cost function helps with saturated output neurons, but not with saturated hidden neurons. To fix this, a much thinner curve in the graph is needed. This can be done by initializing the weights as Gaussian random variables with mean zero and standard deviation $\frac{1}{\sqrt{n}}$ [5]. Shown in Figure 3 [4].

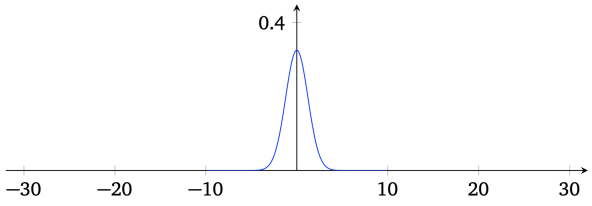


Fig. 3. A normal (Gaussian) distribution with mean 0 and a small standard deviation ($\propto \frac{1}{\sqrt{n}}$), illustrating the spread of the weighted sum z.

Such a neuron is much less likely to saturate, and much less likely to have problems with a learning slowdown. The bias initialization does not have a significant affect on the outcome and does not need the optimizer initialization [6].

III. Implementation

To validate the theoretical knowledge discussed in the previous sections, a basic implementation was developed using Python, along with additional libraries to observe the behavior of the network. The initial model was based on code from the book *Neural Networks and Deep Learning* [4], which was extended with some additional functions to suit this project; see Appendix A. The implementation can be divided into three main parts.

A. Part 1: Data Import and Preparation

The first step involved importing the MNIST dataset, which was used to train and validate the network. The data set was

accessed using the TensorFlow library and contains 70 000 labeled image-label pairs. For this project, 60 000 pairs were used for training, and 10 000 for testing. The training set was then split into 50 000 for training and 10 000 for validation.

B. Part 2: Network Construction

The second part focused on building the network. This involved specifying the number of neurons in the input, hidden and output layers, as well as selecting the appropriate cost function. Since the MNIST data set consists of handwritten digits with the resolution of 28×28 pixels, each image was flattened into a one dimensional vector of 784 values (28×28), which determined the number of input neurons. The output layer contained 10 neurons, one for each digit (0 through 9). The number and size of the hidden layers were fixed according to the original code, which used a single hidden layer containing 30 neurons.

C. Part 3: Training of the Network

Once the network architecture was defined, it was passed to an SGD function for training. Before training could begin, several hyperparameters needed to be set, including the mini-batch size, step size η, and the number of epochs. For this project, the batch size was fixed at 10. However, to improve the results, the step size was somewhat optimized. To determine which step size minimized the cost function more effectively, three values on a logarithmic scale were tested: 0.4, 4.0, and 40. The results of the quadratic cost function are presented in Figure 4, and the cross-entropy cost function is displayed in Figure 9.

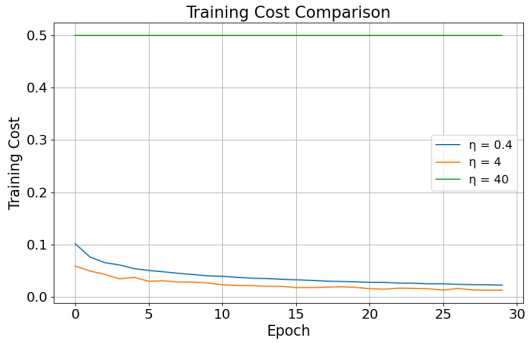


Fig. 4. Training cost for different values of η, when $\lambda = 0$.

The backpropagation algorithm was then used to perform the necessary computations, including the gradient. Together with various helper functions, it calculated and updated the weights and biases of the network to minimize the cost function. After the specified number of epochs, the SGD function returned vectors containing the training cost and the evaluation accuracy for each epoch. These vectors were then used to generate plots for visual understanding of the training process. To avoid retraining the network for further analysis, the trained model was saved as a JSON file.

Figure 5 shows the training cost and validation accuracy for a network trained using the quadratic cost function, while Figure 6, shows the same metrics for a network trained with the cross-entropy cost function. Table I provides a summary of the parameters used in the implementations.

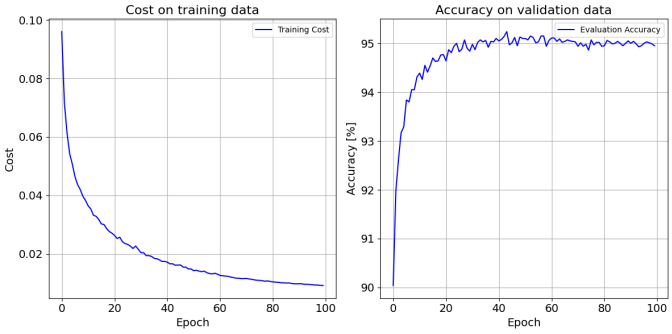


Fig. 5. Network trained using quadratic cost function.

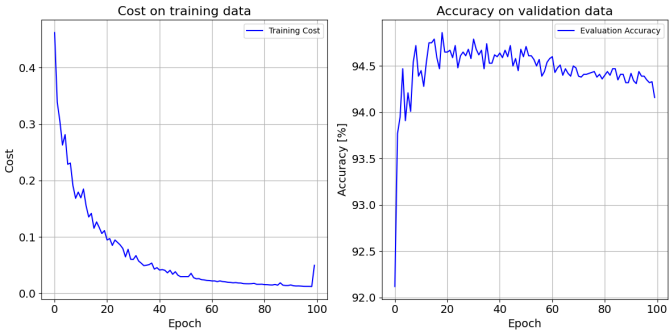


Fig. 6. Network trained using cross-entropy cost function.

After this, a new network was implemented with the addition of a regularization function to prevent saturation. The regularization technique used was the L2 regularization, which required optimization of the regularization parameter λ. Figure 7 illustrates the variation in validation accuracy with different values of λ. The case $\lambda = 0$ serves as a baseline for comparison, as it reflects the validation accuracy of the network without regularization.

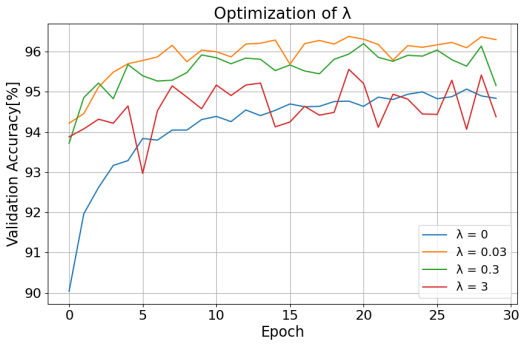


Fig. 7. Validation accuracy for different values of λ.

Once the best performance value of λ was determined, a final adjustment of the step size η was implemented, which can be seen in Figure 8.

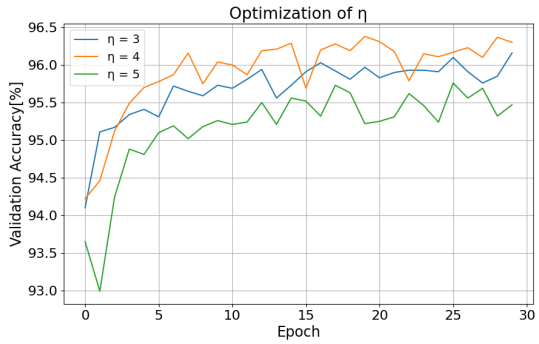


Fig. 8. Final tuning of η with optimized value for λ.

The final network, trained with the tuned parameters, achieved a test accuracy of 96.2%. The hyperparameters used can be seen in Table II.

The same process was then repeated using the cross-entropy cost function to compare its behavior and result with the network trained using the quadratic cost function. The performance of different learning rates is shown in Figure 9.

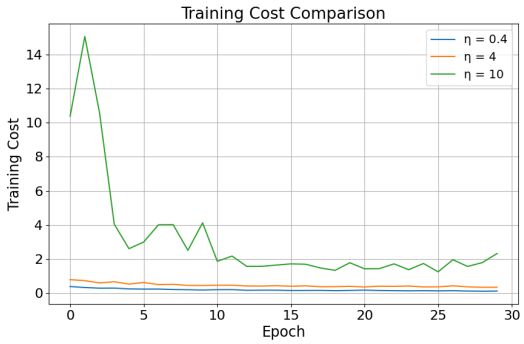


Fig. 9. Training cost for different values of η, when $\lambda = 0$.

Next, the regularization parameter was then applied and optimized by testing three different values, as shown in Figure 10.

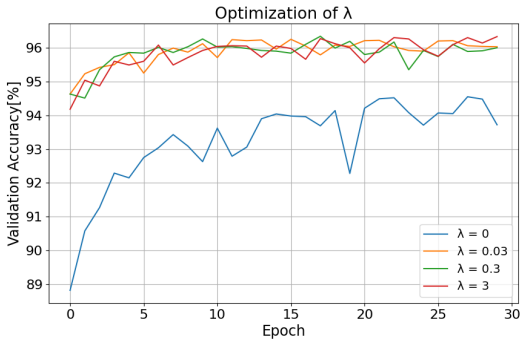


Fig. 10. Validation accuracy for different values of λ.

Finally, the learning rate was fine-tuned, as shown in Figure 11.

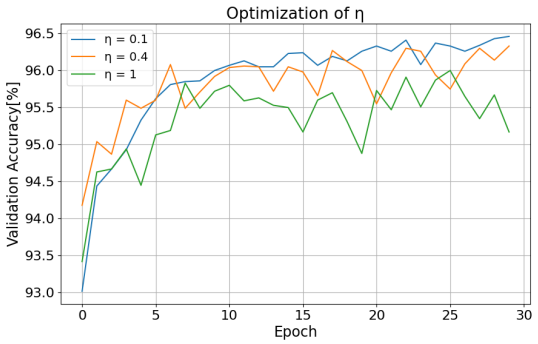


Fig. 11. Final tuning of η with optimized value for λ.

The final network was implemented using these optimized hyperparameters and achieved a classification accuracy of 96.4%. The hyperparameters used in the final network can be seen in Table II.

TABLE I
TABLE OVER PARAMETERS USED TO DISPLAY DIFFERENT COST FUNCTIONS

Network	$C(w,b)$	Epoch	Mini-batch	Step-size η	Reg λ
Fig 5	Quadratic	100	10	4	0
Fig 6	Cross-Entropy	100	10	4	0

TABLE II
TABLE OVER NETWORK PARAMETERS AND FINAL CLASSIFICATION RESULTS

$C(w,b)$	Epoch	Mini-batch	Step-size η	Reg λ	Accuracy
Quadratic	100	10	4	0.03	96.2%
Cross-Entropy	100	10	0.4	3	96.4%

IV. DISCUSSION

The practical implementation of neural networks provided valuable insight into how theoretical concepts such as cost functions, gradient descent, and regularization influence model performance. A series of experiments were conducted to investigate how the choice of cost function, step size, and regularization parameter affect the convergence and generalization performance of the network.

Firstly, the central role of the cost function and how it guides the learning of the network was highlighted. By implementing the network using both the quadratic cost function and the cross-entropy cost function, their difference in convergence speed is clearly visible. As shown in Figure 6, training with the cross-entropy cost function resulted in faster convergence compared to the quadratic cost function (Figure 5). In this experiment, the validation accuracy remained similar for both methods. However, there was a lack of smoothness in the resulting plots when the cross-entropy cost function was used. This is largely a consequence of the inherent randomness of SGD. A way to achieve smoother testing and training curves is to average the results over multiple SGD runs. Nevertheless, the result supports the theoretical understanding

that cross-entropy avoids issues associated with saturated sigmoid activations, thereby enabling more efficient learning. Furthermore, overfitting can clearly be detected when using the cross-entropy cost function due to the drop in accuracy (Figure 6).

A. Quadratic Cost Function Performance

Another key focus was the optimization of the step size η. As shown in Figure 4, different values of the step size had a substantial impact on the cost-minimization process. Although a small step size resulted in slow learning, a larger value resulted in less reduction of the cost. Through testing different values of η, the conclusion that η = 4.0 achieved the most effective cost reduction could be drawn and therefore used in the following implementations.

To address overfitting, L2 regularization was introduced. The regularization parameter λ was optimized by evaluating the validation accuracy for different values. As shown in Figure 7, λ = 0.03 provided the best generalization performance.

Following the selection of λ, a final adjustment of the step size was performed with regularization applied. Figure 8 shows how various sizes influenced performance under regularization, with η = 4 delivering the best results.

The final accuracy achieved was 96.2%. This improvement indicates enhanced generalization performance resulting from the application of regularization and hyperparameter tuning.

B. Cross-Entropy Cost Function Performance

A similar procedure was followed using the cross-entropy cost function. However, the behavior of the network during training exhibited notable differences compared to the quadratic cost function.

As shown in Figure 9, three different learning rates were tested. Both $\eta = 0.4$ and $\eta = 4$ yielded good results. Compared to the quadratic cost function, $\eta = 0.4$ performed better, and the differences in training cost between the learning rates were significantly more pronounced when using the cross-entropy cost function.

Regularization was again introduced to combat overfitting. As shown in Figure 10, the three different values of the regularization parameter λ produced similar results. However, all regularized networks significantly outperformed the unregularized version.

Final tuning of the learning rate under regularization showed that $\eta = 0.1$ provided the smoothest learning curve, while $\eta = 0.4$ offered faster convergence with nearly identical final accuracy. Therefore, $\eta = 0.4$ was selected for the final model, as can be seen in Figure 11.

The final classification accuracy of the network trained with the optimized hyperparameters was 96.4%.

These results indicate that both networks achieved comparable classification performance, suggesting that either cost function can be effective given a suitable hyperparameter optimization. However, if faster learning is a priority, the cross-entropy cost function is preferable because of its more efficient convergence behavior.

C. Hyperparameter

The values in Table II provided a baseline and demonstrated the core behavior of a neural network. However, it should be noted that there are many approaches to determining hyperparameters, such as grid search, random search, manual tuning and so on. The key takeaway is that fine tuning the learning rate, the regularization parameter and the batch size can significantly improve training efficiency. There is no universal rule for optimization and a network can always be optimized further.

V. Conclusion

This project provided a great theoretical understanding of the core concepts in neural networks. A simple implementation was developed to deepen comprehension of the training process.

The results were consistent with theoretical expectations, such as the fact that using the cross-entropy cost function leads to faster convergence compared to the quadratic cost function. Additionally, tuning the step size and regularization parameter significantly improved performance and validation accuracy.

Although the network was not optimized for maximum performance, the implementation successfully demonstrated the fundamental behaviors of neural networks. These findings highlight the importance of selecting and tuning hyperparameters according to the specific task.

VI. Future work

Although other regularization methods such as L1 regularization, dropout and early stopping were discussed in theory, they were not included in the implementation. These could be valuable additions in future work to further analyze overfitting and generalization.

Appendix A
Code for the implementations

Acknowledgment

The authors would like to thank their supervisor Seyedamirreza Kazemi for his support, insightful advice, and the valuable discussions and meetings throughout the course of this project.

References

[1] Y.-c. Wu and J.-w. Feng, "Development and application of artificial neural network," *Wireless Personal Communications*, vol. 102, no. 2, pp. 1645–1656, SEP 2018.

[2] Wikipedia contributors. (2024) Von Neumann Architecture. [Online]. Available: https://en.wikipedia.org/wiki/Von_Neumann_architecture

[3] R. Rojas, *Neural Networks*, 1st ed. Heidelberg, Germany: Springer Berlin, Heidelberg, 1996.

[4] M. A. Nielsen, *Neural Networks and Deep Learning*, 1st ed. San Francisco, USA: Determination Press, 2015.

[5] C. M. Bishop, *Pattern Recognition and Machine Learning*, 1st ed. New York, USA: Springer New York, NY, 2006.

[6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, 1st ed. Massachusetts, USA: MIT Press, 2016.